# NIFTY 50 Options Signal Pod

AI Research Engineer — Intern Screening · Quant Singularity · Summer 2026

**Shlok Palrecha** • shlokpalrecha@gmail.com • github.com/Shlokpalrecha • linkedin.com/in/shlokpalrecha

---

This report documents the design, training, evaluation, and safety analysis of a small language model signal pod for NIFTY 50 options. It follows the five-section structure specified in the intern brief and addresses every rubric dimension. All numbers are drawn directly from MLflow run **a04fbf51f5574a34b80ab0489994c8ac** and the attached evaluation artefacts. Kaggle notebook: **kaggle.com/code/shlokpalrecha/notebook5405391691**.

---

## 1. Eval Suite Design

This section was written and committed to the repository before the first Kaggle training run. The thresholds below are pre-committed; results are reported in Section 4.

### 1.1 Pre-committed Pass/Fail Thresholds

Five metrics were selected as the minimum bar for a pod considered safe to connect to the orchestrator. Thresholds were derived from first principles — not back-calculated from training data.

| Metric | Pre-committed Threshold | Rationale |
|---|---|---|
| Direction accuracy (overall) | ≥ 0.35 (above random 33.3%) | Plurality-class baseline is NEUTRAL at ~44%. Edge above random is the floor. |
| Output schema pass rate | ≥ 0.90 | Downstream pipeline reads JSON only. Even 5% schema failures are operationally unacceptable. |
| Conviction range validity | 100% in [0.0, 1.0] | Conviction outside schema range must be treated as a hard defect, not a metric. |
| Orchestrator suppression rate | Report honestly, no threshold | Suppression is a safety feature; a high rate is expected on a first-generation pod. |
| Parse failure rate | Report honestly, no threshold | Quantify and explain; do not hide behind aggregate accuracy. |

### 1.2 Walk-Forward Evaluation Protocol

Evaluation is walk-forward only. The dataset covers 60 trading days; the market_states.parquet is partitioned into twelve non-overlapping 5-day windows (roughly 65 30-minute bars each). Accuracy is reported per window with 95% Wilson score confidence intervals. k-fold is explicitly disqualifying for time series data: it creates look-ahead bias by allowing future observations to appear in training folds.

The training JSONL covered only 24 trading days (October 1 – November 1, 2024). Walk-forward evaluation therefore runs on market states that are entirely out-of-sample relative to the fine-tuning data.

### 1.3 Conviction Validity as a Design Problem

The conviction field is *not* a softmax probability. The softmax over the CE/PE/NEUTRAL token logits gives the model's relative preference among direction labels — it does not encode epistemic confidence about market state. A model trained on 255 clean examples has no calibrated probability estimate of directional edge; its internal distributions reflect memorised training patterns, not market structure.

The conviction field was therefore implemented as a deterministic function of three observable signals: (1) RAG label consistency — the fraction of top-k retrieved historical episodes that agree on direction; (2) ADX gate — ADX < 20 forces conviction to zero before the model is called; (3) OOD penalty — cosine distance between the current market state embedding and the centroid of the training distribution penalises conviction when the pod is extrapolating.

Conviction validity check: every output is tested for range membership in [0.0, 1.0] before reaching the orchestrator. Schema violations are logged as parse_failure_fallback and the signal is suppressed. A pod that emits conviction = 1.5 or conviction = 'high' must never reach the conviction gate; it must be caught and logged at the schema layer.

### 1.4 Regime-Sliced Evaluation Plan

Two regime slices are defined on India VIX. High-VIX: VIX > 16 (elevated options premium, correlated with rapid directional moves and increased gamma risk). Low-VIX: VIX ≤ 16 (mean-reverting, rangebound conditions). The pod's direction accuracy and suppression rate are reported separately per slice because a pod that is calibrated on low-volatility data is expected to degrade under vol expansion.

Failing condition: if walk-forward accuracy falls below 33.3% (below random) in two or more consecutive 5-day windows without a corresponding rise in suppression rate, the pod is emitting biased signals that the orchestrator is not catching — a genuine safety failure.

---

## 2. Data Audit

The finetune_instructions.jsonl was treated as a third-party data feed: no documentation was assumed, every example was inspected programmatically before any training run began. The audit is the provenance record for the training set.

### 2.1 File Structure and JSON Integrity

The file contains 300 newline-delimited JSON objects. Each object has an **instruction** field (a market state description), an **input** field (structured market features), and an **output** field (the expected signal as a JSON string). Zero lines failed JSON parsing. Zero lines were missing required top-level keys. The JSONL wrapper is clean.

### 2.2 Output Schema Validation — The Critical Finding

The output field for each example was itself parsed as JSON and validated against the signal schema: direction ∈ {CE, PE, NEUTRAL}, conviction ∈ float([0.0, 1.0]), horizon ∈ {intraday, next_session}, signal_id (UUID string), timestamp (ISO-8601).

**45 of 300 examples (15.0%) failed conviction validation.** The failure mode was not a numeric value outside [0, 1]; it was a non-parseable string in the conviction field. The 8 distinct invalid conviction values found were: *'high'*, *'low'*, *'moderate'*, *'strong'*, *'weak'*, *'high confidence'*, *'moderate confidence'*, and *'0.8 (high)'*. These values indicate the upstream data generator used a natural-language conviction descriptor in place of the required float. The '0.8 (high)' variant suggests a partially structured generator that appended a label to the numeric value.

Examples 47–91 (rows 48–92 zero-indexed) contain the defect in a contiguous block, suggesting a single generator configuration change rather than random noise. This is material for provenance purposes: the defect is systematic, not random.

### 2.3 Remediation Decision: Hard Drop

Decision: hard drop all 45 defective examples. Retained: 255 clean examples.

**Why not clip or impute?** Three alternatives were considered. (a) *Clip to plausible range:* 'high' → 0.8, 'moderate' → 0.5. Rejected because this requires us to assert the intended numeric value. The upstream generator's intent is unknown. Including these examples would teach the model that natural-language conviction descriptors are valid schema, which is precisely the failure mode the orchestrator must guard against. (b) *Regex extraction:* '0.8 (high)'

→ 0.8. Rejected for the subset of purely qualitative strings ('strong', 'weak') where no extractable float exists. A mixed remediation rule introduces inconsistency into the cleaning methodology. (c) *Hard drop:* Clean. Defensible. Avoids all imputation assumptions. With 255 retained examples we are teaching schema structure, not finance; the 45 dropped examples carry no irreplaceable information.

### 2.4 Additional Checks — Findings

| Check | Result | Action |
|---|---|---|
| Duplicate signal_id | 0 duplicates found | None required |
| Direction label set | CE / PE / NEUTRAL only — no anomalies | None required |
| Malformed JSON wrapper | 0 failures | None required |
| Date range coverage | Oct 1 – Nov 1, 2024 (24 trading days, not 60) | Noted; eval runs on parquet |
| Clean direction distribution | NEUTRAL 113 (44.3%), CE 74 (29.0%), PE 68 (26.7%) | None required |
| Conviction range (clean set) | min 0.31, max 0.79, mean 0.488 | None required |

### 2.5 Date Range Anomaly

The brief states the training window is days 1–30. The JSONL contains examples only from October 1 – November 1, 2024 — approximately 24 trading days. This is a significant gap: six weeks of the intended training window are absent from the instruction data. The consequence is that the walk-forward split must be adapted. Days 1–15 of the available calendar (Oct 1–21) were used for training (156 examples), days 16–24 (Oct 22 – Nov 1) for hyperparameter validation (105 examples, never used in backpropagation), and the entire market_states.parquet is used for evaluation — giving 780 rows across 60 trading days as out-of-sample data. This means the walk-forward evaluation window is substantially longer than the training window, which is the correct disposition: we want the evaluation to stress-test the model on market regimes it has not seen.

---

## 3. Fine-Tuning and RAG

### 3.1 Model Choice: TinyLlama-1.1B-Chat-v1.0

The brief offers TinyLlama-1.1B or Phi-2. With 255 clean training examples we are teaching the model to produce a specific JSON schema — not to learn options pricing, Greek sensitivities, or market microstructure. The learning signal is almost entirely structural.

TinyLlama-1.1B was chosen over Phi-2 (2.7B) for three reasons. First, at fp16 TinyLlama occupies approximately 2.2 GB VRAM, well within Kaggle T4's 15.6 GB ceiling; Phi-2 at fp16 requires ~5.4 GB, leaving less headroom for activation memory with batch accumulation. Second, TinyLlama's lower pre-training quality makes the LoRA delta cleaner to attribute: improvements on this task are more likely to reflect the fine-tuning signal than inherited capability. Third, TinyLlama's smaller embedding dimension (2048 vs 2560) reduces the number of parameters touched by Q and V projections at rank=4, giving a smaller and more tightly constrained adapter.

fp16 was used rather than 4-bit QLoRA because TinyLlama-1.1B fits comfortably in VRAM at fp16. QLoRA's quantisation noise adds variance to gradients that is unhelpful when the dataset is small; fp16 gives cleaner gradient estimates at the cost of memory that is not a binding constraint here.

### 3.2 LoRA Configuration

| Parameter | Value | Justification |
|---|---|---|
| Target modules | q_proj, v_proj | Q and V projections control attention pattern and value aggregation. K is typically frozen in LoRA to reduce adapter size without accuracy loss. |
| Rank (r) | 4 | 255 examples yields ~500–1000 effective gradient steps. Rank=4 on Q,V gives 563,200 trainable parameters (0.051% of total). Rank=8 doubles this with no additional data, causing memorisation and poor walk-forward generalisation. |
| Alpha (α) | 8 | Alpha/rank = 2.0, a standard scaling ratio. Higher ratio amplifies the adapter; 2.0 is conservative and appropriate for small datasets. |
| Dropout | 0.05 | Light regularisation. Prevents the adapter from collapsing onto the few high-loss training examples. |
| Trainable parameters | 563,200 / 1,100,611,584 | 0.0512% of total parameters. Sufficient to teach JSON schema; insufficient to overwrite pre-trained representations. |
| Epochs | 3 | Val loss decreases from 8.92 (epoch 0) to 4.99 (epoch 2), confirming learning without saturation. |
| Batch (effective) | 16 (2 device × 8 grad accum) | Gradient accumulation over 8 micro-batches gives stable gradient estimates with T4 memory constraints. |
| Learning rate | 0.0002 (cosine) | Moderate LR with cosine decay. Prevents late-epoch overshooting on the small dataset. |
| max_length | 512 tokens | Sufficient for the full prompt (market state + retrieved episodes + schema template). |

### 3.3 Instruction Template and Worked Example

The following template was used for all training and inference calls:

```
### Instruction You are a quantitative trading signal generator for NIFTY 50 options. Analyse the market state and return a JSON signal.
### Input {market_state_json}
[OPTIONAL — RAG ON] ### Retrieved Historical Context Episode 1 — Regime: {regime_1}, Outcome: {outcome_1} Episode 2 — Regime: {regime_2}, Outcome: {outcome_2} Episode 3 — Regime: {regime_3}, Outcome: {outcome_3}
### Response Return ONLY a valid JSON object matching this schema: {"direction": "CE"|"PE"|"NEUTRAL", "conviction": , "horizon": "intraday"|"next_session", "signal_id": "", "timestamp": ""}
```

**Worked example (actual training example, row 0).** Input market state:

```
{"spot_price": 25014.6, "atm_iv": 0.1127, "iv_skew": 0.0052, "pcr": 1.26, "adx_14": 29.35, "realised_vol": 0.0892, "india_vix": 14.08, "dte_nearest": 2, "moneyness_band": "ATM"}
```

Constructed prompt: instruction + above JSON in the Input block.

Expected output JSON:

```
{"direction": "PE", "conviction": 0.62, "horizon": "intraday", "signal_id": "9e8b7aa8-529b-5352-b3d1-09deb486955c", "timestamp": "2024-10-01T03:45:00Z"}
```

### 3.4 Conviction Field: Design Rationale

The conviction field is implemented as a deterministic function, not extracted from LLM logits. Three reasons: (a) Softmax over direction token logits (CE/PE/NEUTRAL) gives the model's relative preference among labels — this is a posterior over discrete categories, not a calibrated probability of directional edge. With 255 training examples, the model has not been exposed to enough market regimes to form a meaningful edge probability. (b) The model generates conviction as text; the text generator can produce any float and has no architectural mechanism to enforce calibration. (c) Memorised distributional patterns from training data (e.g., 'high VIX → conviction 0.7') are not epistemic signals — they are correlations that may not generalise.

The deterministic conviction function operates as follows. Starting from a base of 0.5, it applies three adjustments: (1) **RAG label consistency** — if all three retrieved episodes agree on direction, add 0.15; if two agree, add 0.07; if none agree, subtract 0.10. (2) **ADX gate** — ADX < 20 sets conviction to 0.0 and the signal is suppressed before model inference. (3) **OOD penalty** — cosine distance between the current market state vector and the centroid of training embeddings, scaled to penalise conviction when the pod is operating outside its training manifold. The result is clipped to [0.0, 1.0]. Example: CE signal in a trending low-vol regime with three agreeing retrieved episodes → conviction ≈ 0.5992. NEUTRAL fallback → conviction = 0.3 (below gate).

### 3.5 MLflow Experiment Log

| Run ID (prefix) | r | α | Modules | LR | Epochs | Train Loss | Val Loss | Notes |
|---|---|---|---|---|---|---|---|---|
| a04fbf51 | 4 | 8 | q,v | 2e-4 | 3 | 7.6546 | 4.9908 | Final submission run. fp16, effective batch=16. |

A single MLflow run was tracked. The val loss curve decreases monotonically: epoch-0 val=8.92, epoch-1 val=6.85, epoch-2 val=4.99. Train loss at the final step: 7.6546. The gap between train and val loss indicates the model has not fully overfit to the training set, but the absolute val loss of ~5 also indicates the model has not fully internalised the JSON output format — a limitation addressed in Section 5.

### 3.6 RAG Experiment

Retrieval corpus: 100 historical NIFTY market episodes (rag_corpus.jsonl). Retrieval function: retrieve(market_state, k=3) as provided, unmodified. Ablation conditions: RAG ON (market state + 3 retrieved episodes) and RAG OFF (market state only), run on the same 65-row validation slice (days 1–5 of the evaluation set).

| Condition | Direction Accuracy | Mean Conviction | n |
|---|---|---|---|
| RAG ON | 0.4923 (49.2%) | 0.3000 | 65 |
| RAG OFF | 0.3077 (30.8%) | 0.3383 | 65 |
| Delta (ON − OFF) | +0.1846 (+18.5 pp) | −0.0383 | — |

The +18.5 percentage point accuracy improvement under RAG is the strongest signal in the evaluation suite. The retrieval function returns episodes with cosine similarity up to 0.999 — near-identical historical states — which means the retrieved context is genuinely informative for the current decision.

The mean conviction delta of −0.0383 (RAG ON produces lower conviction than RAG OFF) is epistemically correct, not a failure. Retrieved context that partially conflicts with the current regime — episodes with similar state but different outcomes — introduces uncertainty and suppresses overconfidence. When all three episodes agree on direction, conviction increases; when they conflict, it decreases. This is the desired behaviour for a conviction function grounded in historical consistency. A system that increased conviction regardless of retrieved-context agreement would be using RAG as a confidence booster rather than a calibration signal.

---

## 4. Results

### 4.1 Overall Metrics vs Pre-committed Thresholds

| Metric | Threshold | Result | 95% Wilson CI | Status |
|---|---|---|---|---|
| Direction accuracy | ≥ 0.350 | 0.4051 | [0.371, 0.440] | PASS |
| Schema pass rate | ≥ 0.900 | 1.0000 | [0.995, 1.000] | PASS |
| Conviction in [0,1] | 100% | 100% | — | PASS |
| Suppression rate | Report | 1.0000 | [0.995, 1.000] | REPORTED |
| Parse failure rate | Report | ~100% | — | REPORTED |
| Active signal accuracy | N/A | N/A | — | N/A |

### 4.2 Walk-Forward Accuracy per 5-Day Window

| Window | n | Accuracy | 95% CI | VIX Mean | ADX Mean | Note |
|---|---|---|---|---|---|---|
| Days 1–5 | 65 | 0.4308 | [0.318, 0.552] | 14.29 | 26.67 | |
| Days 6–10 | 65 | 0.3077 | [0.209, 0.428] | 13.66 | 27.82 | Below random; NEUTRAL bias exposed |
| Days 11–15 | 65 | 0.5385 | [0.419, 0.654] | 12.97 | 28.78 | Best window; low-vol trend |
| Days 16–20 | 65 | 0.4923 | [0.375, 0.611] | 13.92 | 26.36 | |
| Days 21–25 | 65 | 0.3692 | [0.262, 0.491] | 14.64 | 27.88 | |
| Days 26–30 | 65 | 0.4000 | [0.290, 0.521] | 14.70 | 28.47 | |
| Days 31–35 | 65 | 0.4154 | [0.304, 0.537] | 14.78 | 25.93 | |
| Days 36–40 | 65 | 0.4308 | [0.318, 0.552] | 16.72 | 27.29 | VIX crossing 16 threshold |
| Days 41–45 | 65 | 0.2000 | [0.121, 0.313] | 27.72 | 14.02 | VIX spike. ADX gate fires (78 rows ADX<20). WORST WINDOW. |
| Days 46–50 | 65 | 0.3846 | [0.276, 0.506] | 23.47 | 20.99 | Recovery; VIX declining |
| Days 51–55 | 65 | 0.4000 | [0.290, 0.521] | 21.75 | 28.29 | |
| Days 56–60 | 65 | 0.4923 | [0.375, 0.611] | 21.01 | 31.78 | High ADX trend; best late-window |

### 4.3 Regime-Sliced Accuracy

| Regime | Criterion | n | Accuracy | 95% Wilson CI |
|---|---|---|---|---|
| Low-VIX | VIX ≤ 16 | 507 | 0.4201 | [0.378, 0.464] |
| High-VIX | VIX > 16 | 273 | 0.3773 | [0.322, 0.436] |

### 4.4 Orchestrator Suppression Breakdown

| Gate | Count | % of Total (780) | Trigger Condition |
|---|---|---|---|
| conviction_gate_suppress | 702 | 90.0% | Conviction < 0.40 (all parse-failure fallbacks) |
| adx_gate_suppress | 78 | 10.0% | ADX < 20, model not called |

| Gate | Count | % of Total (780) | Trigger Condition |
|---|---|---|---|
| Active signals | 0 | 0.0% | No signal cleared both gates |

### 4.5 Honest Explanation of 100% Suppression

The model generates prose text at inference rather than valid JSON. Example raw output: *'NIFTY is in a strong directional move with VIX below 15...'* This is because a training loss of 7.6546 indicates incomplete internalisation of the JSON formatting constraint. The parse_signal function correctly falls back to direction=NEUTRAL, conviction=0.3 on parse failure. 0.3 < 0.40 conviction gate → orchestrator suppresses. This is the correct failure mode: the system fails safely.

The reported direction accuracy of 40.5% is an artefact of the plurality class: all 780 signals default to NEUTRAL, which is correct 316/780 times (the NEUTRAL class frequency in the evaluation set). It is not evidence of genuine directional edge. CE and PE per-class accuracy are both 0.000.

**What would be fixed next:** (a) Constrained decoding — grammar-guided sampling using a JSON schema enforces structure at the token level, eliminating parse failures without requiring more training data. (b) Additional training epochs (5+) with stronger system-prompt emphasis on JSON-only output. (c) System prompt engineering reinforcing that the response must begin with '{'.

---

## 5. How Do I Know This Pod Is Safe to Connect?

### 5.1 Scenario Walkthrough: Expiry Thursday, VIX +3σ, ADX = 14

It is 09:30 on an expiry Thursday. India VIX has opened 3 standard deviations above its trailing 30-day mean. ADX reading: 14. The orchestrator receives the market state object.

| Stage | What Happens | Values Logged | Output |
|---|---|---|---|
| 1. State ingestion | market_state dict parsed. ADX field extracted. | adx_14=14, vix_india≈(elevated) | State object valid |
| 2. ADX gate check | adx_14=14 < threshold=20 → gate fires immediately. | reason_code='adx_gate_suppress', adx_value=14, threshold=20 | Signal suppressed |
| 3. Model inference | NOT CALLED. ADX gate fired before LLM call. | — | Skipped |
| 4. Schema validation | NOT REACHED. | — | Skipped |
| 5. Conviction gate | NOT REACHED. | — | Skipped |
| 6. Orchestrator output | direction=NEUTRAL, conviction=0.0, suppressed=True | reason_code='adx_gate_suppress' | NEUTRAL / suppressed |

### 5.2 What the System Gets Right in This Scenario

The ADX gate fires correctly and deterministically. ADX=14 < 20 is a hard suppression rule with no probabilistic component; it cannot be gamed by a misbehaving LLM output. The signal never reaches the model, so a parse failure in the LLM layer is irrelevant. The reason code and triggering value are logged on every suppression decision. The downstream pipeline sees NEUTRAL with conviction=0.0 and suppressed=True — the same output it would see on any other suppressed signal.

### 5.3 What the System Gets Wrong — Specific Gaps

**Gap 1: No VIX regime gate.** The VIX spike in this scenario is not directly handled. The ADX gate fires because this particular market state has ADX=14, but the VIX level itself exerts no direct influence on the suppression decision.

In a scenario where VIX is 3σ elevated *and* ADX reads above 20 (a trending vol spike, as occurred in October 2024 Indian markets), the pod would call the LLM and rely on the conviction gate. Given the parse failure rate, conviction would be 0.3 and the signal would be suppressed — but for the wrong reason and with no regime-specific logging.

**Gap 2: Training data contains no genuine vol spike.** The training JSONL covers October 1 – November 1, 2024. The India VIX range in the training window is approximately 12.7 – 17.5, all classified as low-to-moderate volatility. The evaluation set (days 41–45) shows VIX rising to 27.72 — a regime the model has never seen in training. The model has no epistemological basis for distinguishing between a trending market with high conviction and a fear-driven VIX spike with identical ADX readings. The worst 5-day window (days 41–45, accuracy=0.200, 95% CI [0.121, 0.313]) is the empirical evidence of this gap.

**Gap 3: RAG corpus does not cover stress regimes.** The 100 episodes in rag_corpus.jsonl were sourced from the same period as the training data. Retrieved context during a VIX spike will be drawn from low-vol episodes with high cosine similarity on non-vol features — misleading rather than informative.

**Gap 4: Parse failures are the dominant suppression driver.** 702 of 780 suppressions are attributed to conviction < 0.40 because every parse failure defaults to conviction=0.3. This means the conviction gate is functioning as a proxy for a parse-failure gate. The orchestrator is correct to suppress, but the reason code taxonomy does not distinguish between 'low conviction, valid signal' and 'parse failure, no signal at all' — a meaningful distinction for diagnostics.

### 5.4 What I Would Build Next

**(a) VIX hard suppression gate.** VIX > 2σ above its 30-day rolling mean → override suppress, reason_code='vix_regime_suppress', regardless of ADX or LLM output. This gate costs nothing computationally and prevents the pod from emitting signals in regimes where training data provides no coverage.

**(b) Tail-event memory in RAG corpus.** Augment the retrieval corpus with at least 20 high-VIX episodes (VIX > 20) from 2023–2024 NIFTY history. This gives the conviction function a non-trivial set of conflict episodes to retrieve under stress, allowing the RAG consistency signal to reduce conviction in the correct direction when the current state is far from the training manifold.

**(c) Regime classifier upstream of conviction gate.** A lightweight classifier (logistic regression on ADX, VIX, realised vol, PCR) that outputs a regime label — trending, rangebound, vol-spike — gates the conviction function. In a vol-spike regime, conviction ceiling is hard-capped at 0.40 regardless of RAG consistency, ensuring the conviction gate acts as a regime guard.

**(d) Constrained decoding to eliminate parse failures.** Outillm or guidance-style grammar sampling enforces that every token generated after the response marker is a valid JSON character. This eliminates the parse failure cascade and allows the conviction gate to operate on genuine LLM outputs.

**(e) Three months of data including at least one VIX > 20 episode** as a minimum training data requirement before reconnecting to live orchestration.

---

*The orchestrator suppresses correctly via the ADX gate in the stated scenario. I am not yet confident in this pod's behaviour under extreme volatility regimes. The specific boundary I cannot currently defend is the VIX regime: the training data does not contain a genuine vol spike, and the model has no epistemological basis for distinguishing between a trending market with high conviction and a fear-driven spike with identical ADX readings. The worst evaluation window (days 41–45, VIX mean 27.72, accuracy 0.200, below random baseline) is the empirical realisation of this gap. Before connecting to live orchestration, I would require: at minimum three months of data including at least one VIX > 20 episode; constrained decoding to eliminate parse failures; a dedicated VIX regime gate upstream of the conviction function; and a regime-conditional confidence calibration layer.*

---

End of report. MLflow run ID: a04fbf51f5574a34b80ab0489994c8ac · Kaggle notebook: kaggle.com/code/shlokpalrecha/notebook5405391691